Table of Contents

4.1 Plan Introduction 2

4.1.1 Project Deliverables 3

4.2 Project Resources 5

4.2.1 Hardware Resources 6

4.2.2 Software Resources 8

4.3 Project Organization 9

4.4 Schedule 11

4.4.1 PERT / GANTT Chart 11

4.4.2 Task / Resource Table 12

4.1 Plan Introduction

This Software Development Plan provides the details of the planned development of RealEye, a Chrome extension designed to help users identify potentially AI-generated content while browsing the web.

RealEye will allow users to scan webpage content for potentially AI-generated text and images and receive a clear visual warning and AI-likelihood result. The system will combine a machine-learning classification model with available content provenance information, such as SynthID and C2PA/Content Credentials, to provide users with multiple indicators rather than relying on a single detection method. The project is intended to make AI-generated content easier to recognize, particularly for users with limited technical experience. Increasingly realistic AI-generated content can make scams, impersonation, and misinformation more difficult to identify.

Development activities will include defining system requirements, designing the user interface and system architecture, collecting and preparing training data, fine-tuning pretrained machine-learning models, developing the Chrome extension and supporting API, integrating the detection components, and performing unit, integration, and system testing. The team will also conduct regular meetings, maintain project documentation, and track development progress throughout the semester. The primary development scope will focus on text and image detection, with additional features considered only if time permits.

Major project milestones include:

- **Project Proposal and Presentation — September 10, 2026 (Week 2):** Complete the initial project proposal outlining RealEye's purpose, scope, planned features, and technologies.
- **Software Requirements Specification and Preliminary Development Schedule — October 1, 2026 (Week 5):** Define the functional and nonfunctional requirements, system components, and initial development schedule.
- **Initial System Architecture, API Contract, and Working Prototype — October 13, 2026 (Week 7):** Establish the project repository structure, define the communication format between the Chrome extension and backend, and implement and test an initial end-to-end prototype using simulated detection results.
- **Software Development Plan — October 15, 2026 (Week 7):** Complete the development plan, including project resources, team responsibilities, major milestones, and the development schedule.
- **Chrome Extension and Backend Development — October 29, 2026 (Week 9):** Expand webpage scanning to support text and images, implement additional user-interface features, and develop backend request validation and detection functionality.
- **Software Design Description: Architecture Section — November 5, 2026 (Week 10):** Complete the architecture section of the Software Design Description, documenting the major system components, their responsibilities, and interactions.
- **Software Design Description: Detailed Section — November 19, 2026 (Week 12):** Complete the detailed Software Design Description, documenting the internal design, interfaces, data processing, and functionality of the major system components.
- **Critical Design Review — (Week 12 or 13):** Present the system's detailed design, demonstrate the current implementation, and review development progress, technical decisions, and remaining work.
- **Machine-Learning Models and Detection Integration — November 24, 2026 (Week 13):** Prepare training and evaluation datasets, fine-tune text and image classification models,

evaluate their performance, and integrate the trained models and available provenance detection methods with the backend API.

- **Complete System Integration — November 26, 2026 (Week 13):** Integrate the Chrome extension, backend API, machine-learning classifiers, and available provenance detection methods into a working application that provides detection results, explanations, and visual warnings.
- **System Testing and User Manual — December 3, 2026 (Week 14):** Complete unit, integration, and system testing, evaluate detection accuracy and accessibility, and finalize the user manual in the project README.
- **Final Project Presentation — December 8, 2026 (Week 15):** Present and demonstrate the completed RealEye application, including its implementation, detection methods, testing results, and known limitations.
- **Final Product Delivery and Documentation — December 14, 2026 (Week 16):** Complete the final code freeze, deliver the RealEye application and source code, and finalize all project documentation in the GitHub repository.
- **Written Status Reports — Every Other Week Beginning Week 7:** Document completed tasks, ongoing development, upcoming milestones, and any technical challenges or delays throughout the semester.

4.1.1 Project Deliverables

The following deliverables will be provided throughout the development of RealEye. These include the required course documentation, major software components, testing results, and the completed application. Each deliverable represents a significant part of the project's development.

- **Project Proposal and Presentation — September 10, 2026 (Week 2)**

  The Project Proposal introduces RealEye, outlines the problem of increasingly realistic AI-generated online content, and explains the motivation for developing a browser-based detection tool. It identifies the project's target users, core features, proposed technologies, and overall development scope. The accompanying presentation communicates the project's goals and planned implementation approach.

- **Software Requirements Specification and Preliminary Development Schedule — October 1, 2026 (Week 5)**

  The Software Requirements Specification (SRS) defines the functional and nonfunctional requirements of RealEye, including webpage scanning, text and image analysis, detection results, visual warnings, and available provenance checks. It also identifies the major system components, their responsibilities, performance requirements, and development and execution environments. The preliminary development schedule establishes the initial timeline for completing the project's major tasks and deliverables.

- **Initial System Architecture, API Contract, and Working Prototype — October 13, 2026 (Week 7)**

  The initial system architecture establishes the organization of RealEye's Chrome extension, backend server, and machine-learning components. The API contract defines the request and response formats used for communication between the extension and server. The working prototype demonstrates the basic end-to-end workflow, allowing users to initiate a scan, extract webpage text, send it to the backend, and view results in the extension popup. At this stage, the prototype uses simulated classification results rather than trained machine-learning models.

- **Software Development Plan — October 15, 2026 (Week 7)**
  The Software Development Plan (SDP) describes how the RealEye project will be developed and managed throughout the semester. It identifies the hardware and software resources, team organization, individual responsibilities, development activities, project deliverables, and major milestones. The document also includes the project schedule and task assignments to guide the team's development and track progress toward completion.

- **Chrome Extension and Backend Development — October 29, 2026 (Week 9)**
  This deliverable expands the initial prototype to support additional webpage scanning and user-interface functionality. The Chrome extension will be developed to extract supported text and images, provide scan controls, display analysis progress, and present detection results and warnings. Backend development will include request validation, input limits, error handling, and the functionality needed to support the detection components. These improvements will establish the foundation for integrating the trained machine-learning models.

- **Software Design Description: Architecture Section — November 5, 2026 (Week 10)**
  The architecture section of the Software Design Description (SDD) will document the overall structure of RealEye and explain how its major components interact. It will describe the Chrome extension interface, webpage interaction functionality, backend API, machine-learning detection system, and evaluation components. The document will also identify the interfaces and communication mechanisms connecting these components.

- **Software Design Description: Detailed Section — November 19, 2026 (Week 12)**
  The detailed section of the Software Design Description will document the internal design of RealEye's major components, including their modules, interfaces, data processing, and interactions. It will provide a more detailed explanation of how the Chrome extension, backend API, machine-learning classifiers, and available provenance detection mechanisms will operate and communicate with one another.

- **Critical Design Review — (Week 12 or 13)**
  The Critical Design Review will present RealEye's detailed system design and demonstrate the functionality implemented up to this stage of development. The team will explain the system architecture, major components, interfaces, and technical decisions, including the integration of the Chrome extension, backend API, and machine-learning detection methods. The review will also address current development progress, known limitations, and any remaining tasks or technical challenges before final system integration and testing.

- **Machine-Learning Models and Detection Integration — November 24, 2026 (Week 13)**
  This deliverable will include the development of machine-learning classifiers for identifying potentially AI-generated text and images. The team will collect and prepare labeled datasets containing human-generated and AI-generated examples, fine-tune appropriate pretrained models, and evaluate their performance using separate validation and testing datasets. The trained models will be integrated with the backend API to replace the simulated classification results. Available C2PA and SynthID detection methods will also be incorporated where technically accessible.

- **Complete System Integration — November 26, 2026 (Week 13)**
  This deliverable will consist of an integrated version of RealEye that connects the Chrome extension, backend API, trained machine-learning classifiers, and available provenance detection mechanisms. The system will allow users to scan supported webpage text and images and receive AI-likelihood estimates, explanations, and visual warnings based on the available detection evidence. The integrated application will provide the core functionality required for final system testing and demonstration.
- **System Testing and User Manual — December 3, 2026 (Week 14)**
  Testing will include unit testing of individual components, integration testing between the Chrome extension and backend, and system testing of the complete application. The machine-learning models will be evaluated using held-out test data, while additional testing will assess functionality, accessibility, performance, and compliance with the SRS requirements. The project's README.md file will also be updated to include a user manual with installation instructions, operating procedures, and troubleshooting information.
- **Final Project Presentation — December 8, 2026 (Week 15)**
  The final presentation will demonstrate the completed RealEye application and explain its development process, system architecture, and implemented features. The team will present the Chrome extension's scanning and detection functionality, discuss the machine-learning models and available provenance checks, and summarize testing and evaluation results. The presentation will also address known limitations and potential future improvements.
- **Final Product Delivery and Documentation — December 14, 2026 (Week 16)**
  The final deliverable will consist of the completed RealEye Chrome extension, backend services, machine-learning components, and associated source code. All project documentation, including the SRS, SDP, SDD, user manual, and testing results, will be finalized and committed to the GitHub repository. The final application will provide the core functionality described in the project requirements, with any incomplete or technically unavailable detection methods clearly documented.
- **Written Status Reports — Every Other Week Beginning Week 7**
  Written status reports will document the team's progress throughout development, including completed tasks, ongoing work, upcoming milestones, and any technical challenges or delays. These reports will help track the project's progress against the development schedule and communicate changes or issues to the instructor. The reports will be maintained in the project's GitHub repository.

4.2 Project Resources

The development of RealEye will require hardware and software resources to support implementation, machine-learning model training, testing, integration, documentation, and demonstration of the completed application. These resources will support the Chrome extension, backend REST API, machine-learning classification models, and available content provenance detection methods. The following sections identify the hardware and software resources required for development and execution of the system.

4.2.1 Hardware Resources

The following hardware resources will be used during the development and execution of RealEye.

Hardware Resource	Development	Execution / Demonstration
Team Member Laptop Computers	X	X
Backend Server / Application Host	X	X
Machine-Learning Training Resources	X	
Test Computers / Devices	X	X
Internet Connection / Network Equipment	X	X

Team Member Laptop Computers

Each team member's personal laptop will serve as a primary development machine for RealEye. These computers will be used to write and debug code, build the Chrome extension, run the backend server locally, develop the user interface, access the GitHub repository, and prepare project documentation. During development, the backend will run locally on a team member's computer using Python, with the Chrome extension communicating with the server through localhost.

The development environment will support macOS and Windows computers, as documented in the project's setup instructions. Each laptop must have sufficient processing power, memory, and storage to run Google Chrome, a code editor, Node.js, Python, and the required development dependencies.

Hardware Specification	Development Computers
Computer	Apple MacBook Air/Pro, Dell, HP, Lenovo, or other Windows laptops
Processor	Apple M1–M5 series, Intel Core i5/i7, or AMD Ryzen 5/7
Memory (RAM)	8–16 GB RAM or greater
Storage	256–512 GB SSD or greater
Operating System	macOS or Windows 11
Network Connectivity	Wi-Fi or Ethernet

Backend Server / Application Host

RealEye will require a backend server to process analysis requests received from the Chrome extension. During initial development and testing, this server will run locally on a team member's laptop. For the final application, the team plans to deploy the backend to a Linux-based host accessible through HTTPS.

The server will execute the FastAPI application, validate incoming requests, communicate with the machine-learning classification models, perform available provenance checks, and return analysis results to the Chrome extension. The selected host must provide sufficient CPU, memory, and storage resources to support the deployed models and handle analysis requests within the performance requirements defined in the SRS.

- **Development host:** Team member's MacBook Pro or Windows laptop running the local FastAPI server
- **Deployment provider:** Render (planned)
- **Operating system:** Linux-based hosting environment
- **Processor:** 2 virtual CPUs (proposed; final allocation dependent on machine-learning model requirements)
- **Memory (RAM):** 4 GB
- **Storage:** 20 GB persistent storage
- **Network connectivity:** Internet connection with HTTPS support for communication with the Chrome extension
- **Server architecture:** 64-bit processor architecture compatible with Python and the required machine-learning libraries

Machine-Learning Training Resources

Machine-learning development will require computing resources for preparing datasets, fine-tuning pretrained text and image classification models, and evaluating model performance. The team will use available development computers.

Model training may benefit from GPU acceleration, particularly for image classification. The exact hardware requirements will depend on the pretrained models selected, dataset sizes, and training configurations. The team will evaluate available computing resources before selecting a final training environment.

- **Training environment:** Team members' computers or an additional computing environment, if needed
- **Storage:** Local SSD storage for training datasets, pretrained models, model checkpoints, and evaluation results

Test Computers / Devices

Multiple computers and browser environments will be used, where available, to verify that RealEye functions consistently across supported development and execution environments. Testing will focus on Google Chrome, since RealEye is designed as a Chrome extension.

These devices will be used to evaluate extension installation, webpage scanning, user-interface behavior, backend communication, accessibility, and system performance. Testing on different computers will help identify issues that may not appear in a single development environment.

Hardware Specification	macOS Test Computer	Windows Test Computer
Device	Apple MacBook Pro	Dell, HP, Lenovo, or other Windows laptop/desktop
Processor	Apple M5 Pro chip	Intel Core i5/i7 or AMD Ryzen 5/7
Memory (RAM)	24 GB unified memory	8–16 GB RAM or greater
Storage	Internal SSD	256–512 GB SSD or greater
Operating System	macOS Tahoe 26.6.2	Windows 11
Browser	Google Chrome 154.0 or later	Google Chrome 154.0 or later
Network Connectivity	Wi-Fi	Wi-Fi or Ethernet

Internet Connection / Network Equipment

Internet connectivity will be required for downloading development dependencies, accessing pretrained machine-learning models and datasets, collaborating through GitHub, and testing RealEye against supported webpages. The final application will also require network connectivity to communicate with the deployed backend server.

Development computers and demonstration devices will use their available network connections. The backend host must support secure HTTPS communication with the Chrome extension.

- **Network connection:** Wi-Fi or wired Ethernet
- **Wireless networking:** Wi-Fi 5 (802.11ac) or newer
- **Network equipment:** Wireless routers, campus networking infrastructure, or other available internet connections
- **Server communication:** HTTPS for secure communication between the Chrome extension and deployed backend

4.2.2 Software Resources

The following software resources will be used to develop, test, deploy, and execute RealEye. The project will primarily use TypeScript and React for the Chrome extension, Python and FastAPI for the backend, and PyTorch and Hugging Face Transformers for machine-learning development.

Software	Version	Purpose
macOS	Tahoe 26.6.2 or later	Development operating system
Windows	Windows 10 or 11	Development and testing on Windows computers
Google Chrome	154.0 or later	Runs and tests the Chrome extension
Chrome Extension APIs	Manifest V3	Browser integration, permissions, and webpage interaction
Visual Studio Code	1.141.1 or later	Primary code editor
Node.js	22.x or later compatible LTS	JavaScript runtime and extension build tools
React	18.3.1	Chrome extension user interface
TypeScript	5.6.3	Extension development
Vite	5.4.11	Builds and bundles the extension
Python	3.11.x	Backend development and ML integration
FastAPI	0.115.6	Backend REST API framework
Uvicorn	0.32.1	Runs the FastAPI server
PyTorch	2.10.0 (planned)	Training and running ML classifiers
Hugging Face Transformers	Compatible release, 4.x or later	Pretrained text and image models
Git	2.5.6	Version control
GitHub	Web-based service	Repository hosting and team collaboration
SQLite	SQLite 3 (planned)	Temporary storage for shared analysis reports
Chrome DevTools	Bundled with Chrome	Debugging and testing the extension

RealEye will use Google Chrome and Chrome Extension APIs (Manifest V3) to interact with webpage content. The Chrome extension will be developed using TypeScript, React, and Vite, with Node.js supporting the development and build process. The backend will use Python, FastAPI, and Uvicorn to process analysis requests and communicate with the detection components. Visual Studio Code, Git, and GitHub will support development and team collaboration.

Machine-learning development will use PyTorch and Hugging Face Transformers to fine-tune and evaluate pretrained text and image classification models. The system will also incorporate C2PA/Content Credentials and SynthID detection tools where technically accessible. Specific tools for these detection methods have not yet been finalized, and unavailable detection methods will be reported accordingly.

SQLite is planned for temporarily storing shareable analysis reports with a seven-day expiration period. Chrome DevTools and additional Python and JavaScript/TypeScript testing tools will support debugging and testing. The backend is planned for deployment to a Linux-based cloud hosting environment using Render, with final configurations and software dependencies determined during development.

4.3 Project Organization

The RealEye development team consists of five members who will collaborate on the design, implementation, testing, documentation, and delivery of the application. The following table identifies the major project tasks and their assigned team members. Responsibilities will be distributed based on each member's experience and availability, with collaboration across tasks as needed.

Task	Assigned Team Member(s)
Project Planning and Documentation	
Project proposal and initial presentation	Everyone
Software Requirements Specification (SRS) and preliminary schedule	Noa, Tara
Software Development Plan (SDP)	Charlize, Tara
Software Design Description (SDD) — architecture section	Everyone
Software Design Description (SDD) — detailed design section	Everyone
Critical Design Review preparation and presentation	Everyone
Biweekly status reports and progress tracking	Everyone
Project coordination, meetings, and task management	Everyone
Chrome Extension and User Interface	
Chrome extension setup and configuration	Charlize
Webpage text extraction and scanning	Charlize
Webpage image extraction and scanning	Charlize
Full-page and selected-area scanning	Charlize
Multiple API requests for scans exceeding 10 items	Charlize
Scan progress indicators and cancellation functionality	Charlize
Results panel displaying AI-likelihood estimates and detection evidence	Charlize

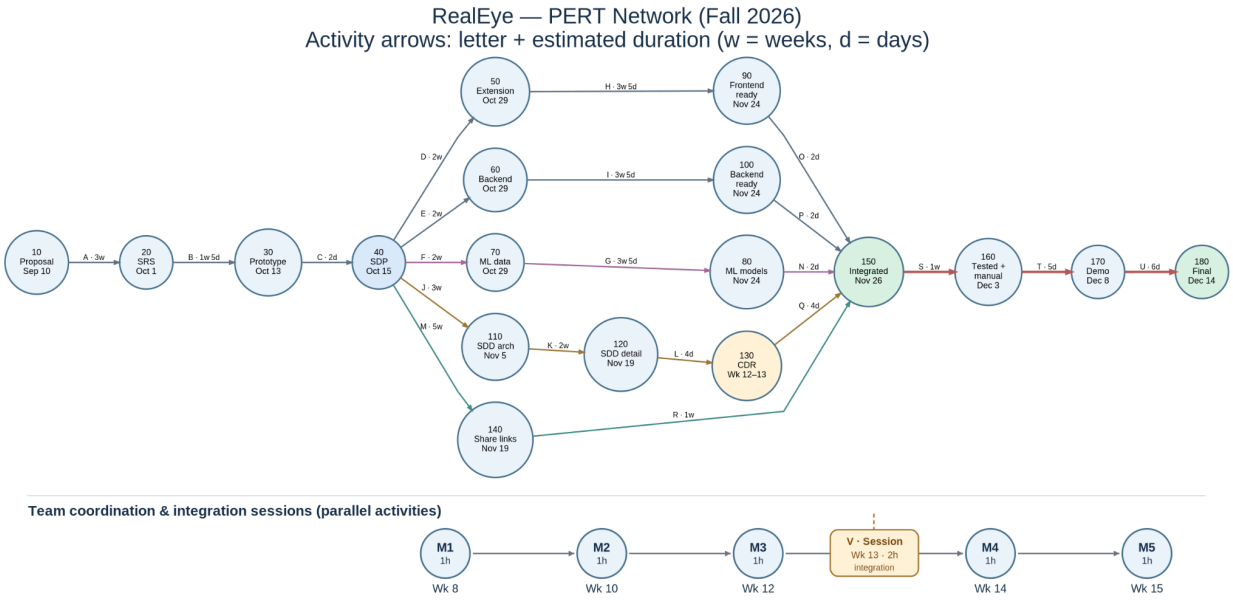
Task	Assigned Team Member(s)
Inline webpage highlights and warning labels	Charlize
Help view, accessibility, and user-interface improvements	Charlize
Backend Server and API	
Backend setup and REST API development	Anastasia
API request and response validation	Anastasia
Text and image input validation, size limits, and error handling	Anastasia
Request rate limiting and origin verification	Anastasia
Detection warning rules and combined analysis results	Anastasia
C2PA/Content Credentials detection integration	Anastasia
SynthID detection integration, where accessible	Anastasia
Machine-Learning Development	
Research and selection of pretrained text and image models	Noa, Tara
Collection and preparation of labeled datasets	Noa, Tara
Dataset licensing review and training/validation/test splits	Noa, Tara
Fine-tuning of text classification model	Noa, Tara
Fine-tuning of image classification model	Noa, Tara
Selection of classification thresholds using validation data	Noa, Tara
Evaluation of model accuracy, F1 score, and false-positive rate	Noa, Tara
Integration of trained models with backend API	Noa, Tara
Additional Features and Deployment	
Shareable analysis report links and copy functionality	Anastasia
SQLite database implementation and seven-day report expiration	Anastasia
Backend deployment to cloud hosting provider	Anastasia
Chrome extension packaging and final configuration	Everyone
Testing, Integration, and Final Delivery	
Unit testing of extension, backend, and detection components	Everyone
Integration testing between extension, backend, and ML models	Everyone
System testing against SRS functional and performance requirements	Everyone
Accessibility, keyboard navigation, and usability testing	Charlize
Bug fixing, debugging, and final system improvements	Everyone
User manual and installation instructions	Charlize
Final project poster and demonstration	Everyone
Final source code, documentation, and project delivery	Everyone

4.4 Schedule

This section provides schedule information for the RealEye project. The development process, tasks, and timeline are detailed in the following subsections.

4.4.1 PERT / GANTT Chart

The following PERT chart illustrates the planned development schedule for RealEye, including major project activities, estimated task durations, dependencies, and completion milestones. Tasks are organized to show which activities must be completed sequentially and which can be performed in parallel. The chart includes the development of the Chrome extension, backend server, machine-learning models, system integration, testing, and final project delivery.



Activity	Task Description
A	Define requirements and complete SRS
B	Develop initial system architecture, API contract, and working prototype
C	Complete Software Development Plan (SDP)
D	Implement image extraction, batch scanning, and scan controls
E	Implement backend input validation, error handling, and security measures
F	Collect, prepare, and split machine-learning datasets
G	Fine-tune and evaluate text and image classification models
H	Develop results panel, inline warnings, and help interface
I	Implement C2PA checks, warning rules, and ML API integration
J	Complete SDD architecture section
K	Complete SDD detailed design section
L	Prepare and conduct Critical Design Review
M	Implement SQLite database and shareable result links
N	Integrate ML models with the backend, including integration checks
O	Integrate Chrome extension components and verify frontend communication

Activity	Task Description
P	Integrate backend components and verify API communication
Q	Address Critical Design Review findings
R	Integrate shareable report functionality
S	Perform system testing, accessibility testing, and complete user manual
T	Prepare and conduct final project demonstration
U	Complete final fixes, packaging, deployment, and project delivery
V	Conduct coordinated frontend, backend, and ML integration session (Week 13)

Team meetings

Meeting	Week	Duration	Planned Discussion
M1	Week 8	1 hour	Review prototype and assign development tasks
M2	Week 10	1 hour	Discuss extension, backend, ML progress, and architecture
M3	Week 12	1 hour	Review detailed design and coordinate integration
M4	Week 14	1 hour	Discuss testing results, bugs, and final improvements
M5	Week 15	1 hour	Review presentation and final delivery preparation

4.4.2 Task / Resource Table

The following table summarizes the people, hardware, and software resources allocated to each major project workstream.

Task Category	People	Hardware	Software
Project Planning and Documentation	Everyone (reviews, meetings, reports)	Team laptops; internet connection	GitHub; documentation and presentation software
Chrome Extension and User Interface	Charlize (lead)	Team laptops; Chrome-compatible test computers	Google Chrome; Chrome Extension APIs (Manifest V3); TypeScript; React; Vite; VS Code; GitHub; Chrome DevTools
Backend Server and API	Anastasia (lead); Noa, Tara (ML integration)	Team laptops; local backend server; planned cloud host	Python; FastAPI; Uvicorn; VS Code; GitHub; C2PA and SynthID tools (where accessible)
Machine-Learning Development	Noa, Tara (text and image models)	Team laptops; machine-learning training resources	Python; PyTorch; Hugging Face Transformers; VS Code; GitHub
Additional Features and Deployment	Anastasia (backend/ deployment); Charlize (extension/sharing UI)	Team laptops; planned Render cloud host; test computers	SQLite; Render (planned); Chrome Extension APIs; Vite; GitHub
Testing, Integration, and Final Delivery	Everyone (shared)	Team laptops; test computers; backend server; internet connection	Google Chrome; Chrome DevTools; GitHub; Python and JavaScript/TypeScript testing tools; documentation and presentation software